Matemática para Aprendizaje Automático 2025

Ing. en Inteligencia Artificial y Ciencia de Datos - Primer Parcial

Instrucciones Generales

- El parcial tiene una duración de 2.5 horas.
- Se permite el uso de calculadora, pero no de dispositivos electrónicos (celulares, tablets, etc.).
- La prueba es estrictamente individual. No se permite el uso de apuntes, libros o cualquier otro material.
- Todas las respuestas deben ser justificadas. Las respuestas sin justificación no serán consideradas.

Pregunta 1 (6 puntos)

Sea el siguiente código en Python:

```
import numpy as np
X = np.array([[1,2,3],[4,5,6],[7,8,9]])
v = np.array([1,2,3])
Y = X+v
```

De el valor final de la variable Y. ¿Qué tipo de objeto es?

Pregunta 2 (6 puntos)

Sea el siguiente código en Python:

```
import numpy as np
X = np.array([[1,2,3,4,5],[4,5,6,7,8],[7,8,9,10,11],[10,11,12,13,14],[13,14,15,16,17]])
v = X[:,1]
x = v[v<14]
z = x[1:3]
```

Escriba los valores que toman las variables v, x y z. ¿Qué tipo de objetos son?

Pregunta 3 (6 puntos)

El siguiente código en Python calcula una operación básica entre dos matrices mediante bucles `for`, indique cuál es esta operación y dé un código que realice la misma operación de forma más eficiente.

```
import numpy as np
def matrix_operation(A,B):
    m,n = A.shape
    p,q = B.shape
    if n != p:
        raise ValueError("Las matrices no son compatibles para la operación.")
    C = np.zeros((m,q))
    for i in range(m):
        for j in range(q):
            C[i,j] = 0
            for k in range(n):
                C[i,j] += A[i,k]*B[k,j]
    return C
```

Pregunta 4 (6 puntos)

Se implementa la suma de dos matrices A y B de tamaño 1000x1000 en algún sistema de cómputo. Se observa que la operación toma 0.1 segundos. Se desea a continuación realizar la misma operación con matrices cuadradas de tamaño 2000x2000. Estime el tiempo que tomará esta segunda operación.

Pregunta 5 (6 puntos)

Sea un vector y de largo m, una matriz A de tamaño $m \times n$ y un vector x de largo n. ¿Cuántas operaciones se requieren para calcular el producto $y^T Ax$? ¿Importa el orden de las operaciones? En el caso de que importe, determine el orden óptimo.

Pregunta 6 (6 puntos)

Defina, en sus palabras, el concepto de número de condición de un problema. Calcule el número de condición de la operación raíz cuadrada.

Pregunta 7 (9 puntos)

Se resuelve el siguiente sistema de ecuaciones lineales utilizando punto flotante de 32 bits:

$$\begin{bmatrix} 8 & 0 & 4 \\ 0 & 400 & 0 \\ 4 & 0 & 8 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = b$$

Donde b es un vector conocido, y x es la incógnita. Estime el error relativo máximo que puede tenerse en la solución si se usa un algoritmo estable.

Problema (15 puntos)

Sea la matriz de datos:

$$X = \begin{bmatrix} 1 & 8 & 27 & 64 & 125 \\ 1 & 2 & 3 & 4 & 5 \\ 1 & 1 & 1 & 1 & 1 \end{bmatrix}$$

Dicha matriz representa un conjunto de 5 datos en 3 dimensiones. Se desea aplicar PCA a estos datos.

1. Calcule las primeras dos componentes principales de los datos.
2. ¿Cuál es la varianza explicada por la primera componente principal respecto de la varianza total de los datos?
3. Si los datos centrados se reconstruyen a partir de las dos primeras componentes principales, ¿cuál es la norma Frobenius del residuo?

Soluciones

Pregunta 1

La operación `Y=X+v` utiliza broadcasting para sumar el vector `v` (como fila) a cada fila de la matriz `X`. El resultado es una matriz de tamaño 3x3:

$$Y = \begin{bmatrix} 2 & 4 & 6 \\ 5 & 7 & 9 \\ 8 & 10 & 12 \end{bmatrix}$$

El tipo de objeto es un `numpy.ndarray` de tamaño 3x3, que es una matriz de 3 filas y 3 columnas.

Pregunta 2

La primera línea del código define una matriz `X` de tamaño 5x5:

$$X = \begin{bmatrix} 1 & 2 & 3 & 4 & 5 \\ 4 & 5 & 6 & 7 & 8 \\ 7 & 8 & 9 & 10 & 11 \\ 10 & 11 & 12 & 13 & 14 \\ 13 & 14 & 15 & 16 & 17 \end{bmatrix}$$

La segunda línea define un vector `v` que es la segunda columna de `X`, pues Python usa indexación comenzada en 0:

$$v = \begin{bmatrix} 2 & 5 & 8 & 11 & 14 \end{bmatrix}$$

Notar que en Python los vectores como este no son ni filas ni columnas, sino un vector de una dimensión.
La tercera línea define un vector `x` que es el resultado de filtrar `v` para obtener solo los elementos estrictamente menores a 14.

$$x = \begin{bmatrix} 2 & 5 & 8 & 11 \end{bmatrix}$$

La cuarta línea define un vector `z` que es el resultado de tomar los elementos de `x` desde el segundo hasta el cuarto (sin incluir el cuarto):

$$z = \begin{bmatrix} 5 & 8 \end{bmatrix}$$

El tipo de objeto de `v`, `x` y `z` es un `numpy.ndarray` de una dimensión (vectores). En este caso, son vectores de tamaño 5, 4 y 2 respectivamente.

Pregunta 3

El código provisto implementa una multiplicación matricial. Esto se desprende directamente de las últimas líneas, incluyendo el último bucle `for` donde se puede ver que se realiza la asignación:

$$C_{ij} = \sum_{k=0}^{n-1} A_{ik} B_{kj}$$

Las líneas precedentes chequean el tamaño de las matrices A y B para verificar que sean compatibles, y en caso contrario se

lanza un error. Luego se inicializa la matriz C como una matriz nula de tamaño apropiado, y finalmente se recorren todos los valores de i y j para calcular los valores de C_{ij}.
Una forma eficiente de realizar la misma operación en `numpy` es mediante `np.dot` o `@`:

```
import numpy as np
def matrix_operation(A,B):
    return A @ B
```

Pregunta 4

El costo computacional de la suma de matrices cuadradas es $O(n^2)$, donde n es el tamaño de la matriz. Por lo tanto, si la suma de matrices de tamaño 1000x1000 toma 0.1 segundos, la suma de matrices de tamaño 2000x2000 tomará aproximadamente 4 veces más tiempo, es decir, 0.4 segundos.

Pregunta 5

El número de operaciones para calcular el producto $c = y^T Ax$ depende del orden. Las dos opciones son: $c = (y^T A)x$ y $c = y^T(Ax)$. En ambos casos se comienza por realizar un producto matriz-vector y un producto escalar (interno) entre vectores.

Si se calcula primero $y^T A$, se realizan n productos internos entre vectores de largo m. Esto implica mn productos escalares y $n \cdot (m-1)$ sumas. Si consideramos que las sumas y productos son operaciones separadas, el costo total es $mn + n(m-1) = mn + nm - n = 2mn - n$. Luego se tiene un costo adicional de $2n - 1$ (n productos y $n-1$ sumas) para calcular el producto interno entre el vector resultante y x. Por tanto, el costo final es $2mn + n - 1$.

Si las operaciones se realizan en el orden opuesto, por simetría, el resultado final es $2mn + m - 1$. En este caso se realizan mn productos y $m(n-1)$ sumas para el producto Ax y $2m-1$ para el producto interno entre y^T y el resultado. Por tanto, el costo total es $2mn + m - 1$.

Puede verse que si $m < n$ conviene hacer la operación en el segundo orden, mientras que si $m > n$ conviene la primera alternativa. En general, el orden óptimo es calcular primero el producto matriz-vector que involucra al vector de mayor tamaño, seguido por el producto escalar. Debe notarse que de todas formas el costo computacional es de orden $O(mn)$, por lo que para matrices grandes la diferencia es despreciable.

Pregunta 6

El número de condición de un problema es una medida de la sensibilidad de la solución del problema ante cambios en los datos de entrada. Se puede utilizar para estimar la propagación de errores de la entrada (argumento) del problema hacia la salida o solución.

En el caso de la operación raíz cuadrada, por ser una operación escalar, el número de condición puede calcularse como:

$$\kappa(\sqrt{x}) = \frac{|x|}{|f(x)|}|f'(x)|$$

En este caso:

$$f(x) = \sqrt{x} \Rightarrow f'(x) = \frac{1}{2\sqrt{x}}$$

Por lo tanto:

$$\kappa(\sqrt{x}) = \frac{|x|}{|\sqrt{x}|} \cdot \left|\frac{1}{2\sqrt{x}}\right| = \frac{1}{2}$$

Pregunta 7

Al usar una representación de punto flotante de 32 bits, la representación de los datos del problema tendrán una precisión relativa de aproximadamente 10^{-7}.
La precisión relativa del resultado, por definición, es el número de condición del problema multiplicado por la precisión relativa de los datos. En el caso de un sistema de ecuaciones lineales, la condición del problema está dado por el número de condición de la matriz del sistema. A su vez, el número de condición de una matriz es:

$$\kappa(A) = \|A\| \cdot \|A^{-1}\| = \sqrt{\frac{\lambda_{MAX}(A^T A)}{\lambda_{MIN}(A^T A)}}$$

En este caso:

$$A^T A = \begin{bmatrix} 8 & 0 & 4 \\ 0 & 400 & 0 \\ 4 & 0 & 8 \end{bmatrix}^T \begin{bmatrix} 8 & 0 & 4 \\ 0 & 400 & 0 \\ 4 & 0 & 8 \end{bmatrix} = \begin{bmatrix} 80 & 0 & 64 \\ 0 & 160000 & 0 \\ 64 & 0 & 80 \end{bmatrix}$$

Un valor propio de esta matriz es 16000. Los otros valores propios pueden hallarse mediante el polinomio característico:

$$\lambda^2 - 160\lambda + 6400 - 4096 = 0$$

Utilizando la conocida fórmula para las raíces:

$$\lambda_{1,2} = \frac{160 \pm \sqrt{160^2 - 4(2304)}}{2} = \frac{160 \pm 128}{2} = \{144, 16\}$$

Por lo tanto: $\kappa(A) = \sqrt{\frac{160000}{16}} = 100$
Esto implica que el error relativo máximo esperado en la solución será del orden de $10^{-7} \cdot 100 = 10^{-5}$.

Problema

Parte 1:
Para calcular las componentes principales, primero se debe centrar la matriz de datos. Esto se hace restando la media de cada columna a cada elemento de la columna. La media de los datos es:

$$\bar{x} = \frac{1}{5} \begin{bmatrix} 1+8+27+64+125 \\ 1+2+3+4+5 \\ 1+1+1+1+1 \end{bmatrix} = \frac{1}{5} \begin{bmatrix} 225 \\ 15 \\ 5 \end{bmatrix} = \begin{bmatrix} 45 \\ 3 \\ 1 \end{bmatrix}$$

La matriz de datos centrados es:

$$X_c = X - \bar{x} = \begin{bmatrix} -44 & -37 & -18 & 19 & 80 \\ -2 & -1 & 0 & 1 & 2 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

El siguiente paso es calcular la matriz de covarianza:

$$C = \frac{1}{5} X_c^T X_c = \frac{1}{5} \begin{bmatrix} -44 & -2 & 0 \\ -37 & -1 & 0 \\ -18 & 0 & 0 \\ 19 & 0 & 0 \\ 80 & 0 & 0 \end{bmatrix} \begin{bmatrix} -44 & -37 & -18 & 19 & 80 \\ -2 & -1 & 0 & 1 & 2 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix} = \begin{bmatrix} 2078 & 60.8 & 0 \\ 60.8 & 2 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

Un valor propio evidente es 0. Los otros dos valores propios son las soluciones del polinomio:

$$\lambda^2 - (2078 + 2)\lambda + 2 * 2078 - 60.8^2 = 0$$

Utilizando la conocida fórmula para las raíces:

$$\lambda_{1,2} = \frac{2080 \pm \sqrt{2080^2 - 4(2078 \cdot 2 - 60.8^2)}}{2} = 1040 \cdot \left(1 \pm \sqrt{1 - \frac{(4156 - 3696.64)}{1040^2}}\right)$$

$$1040 \cdot \left(1 \pm \sqrt{1 - \frac{459.36}{1040^2}}\right) \approx 1040 \cdot \left(1 \pm \sqrt{1 - \frac{0.45}{1040}}\right) \approx 1040 \cdot \left(1 \pm \left(1 - \frac{0.225}{1040}\right)\right) = 1040 \pm (1040 - 0.225)$$

De donde surge que las raíces son $\lambda_1 \approx 2079.775$ y $\lambda_2 \approx 0.225$. Las componentes principales son los vectores propios asociados a los valores propios. La primera componente principal es el vector propio asociado al valor propio 2079.78, que puede hallarse como:

$$(C - \lambda_1 I)v = 0 \Rightarrow \begin{bmatrix} -1.775 & 60.8 & 0 \\ 60.8 & -2077.775 & 0 \\ 0 & 0 & -2079.78 \end{bmatrix} \begin{bmatrix} v_A \\ v_B \\ v_C \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}$$

Es inmediato que $v_C = 0$. Imponiendo $v_1 = 1$ se obtiene:

$$-1.775 + 60.8 v_B = 0 \Rightarrow v_B = \frac{1.775}{60.8} \approx 0.0292$$

Normalizando el vector obtenido $\hat{v}_1 = \frac{v_1}{\|v_1\|}$ se obtiene: $\hat{v}_1 \approx \begin{bmatrix} 0.9996 \\ 0.0292 \\ 0 \end{bmatrix}$

Repetimos el procedimiento para el segundo valor propio:

$$(C - \lambda_2 I)v = 0 \Rightarrow \begin{bmatrix} 2077.775 & 60.8 & 0 \\ 60.8 & 1.775 & 0 \\ 0 & 0 & -2079.78 \end{bmatrix} \begin{bmatrix} v_A \\ v_B \\ v_C \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}$$

De donde nuevamente surge $v_C = 0$. Imponiendo esta vez $v_B = 1$ y usando la segunda ecuación en lugar de la primera obtenemos una ecuación muy similar a la anterior para v_A:

$$60.8 v_A + 1.775 = 0 \Rightarrow v_A = -\frac{1.775}{60.8} \approx -0.0292$$

Normalizando el vector obtenido $\hat{v}_2 = \frac{v_2}{\|v_2\|}$ se obtiene: $\hat{v}_2 \approx \begin{bmatrix} 0.0292 \\ -0.9996 \\ 0 \end{bmatrix}$

Alternativamente, este vector podía obtenerse imponiendo la condición de ortogonalidad con el v_1 hallado anteriormente y con el vector propio asociado a $\lambda_3 = 0$, que es simplemente $v_3^T = \begin{bmatrix} 0 & 0 & 1 \end{bmatrix}$.

Parte 2:

A partir de los resultados vistos en el curso sabemos que la varianza explicada por las primeras k componentes principales (relativo a la varianza total) es:

$$\frac{\sum_{i=1}^{k} \lambda_i}{\sum_{i=1}^{n} \lambda_i}$$

En este caso para $k = 1$ resulta en:

$$\frac{\lambda_1}{\lambda_1 + \lambda_2 + \lambda_3} = \frac{2079.775}{2080} \approx 0.9999$$

Esto significa que la primera componente principal explica el 99.99% de la varianza total de los datos.

NO SE PEDÍA: Este fenómeno sugiere que los datos están 'casi' en una misma línea. En realidad, si le llamamos x, y, z a las coordeneadas de los datos, estos están exactamente en la curva $x = y^3$. Sin embargo, dados los puntos que se tienen, a todos los efectos prácticos la primera componente varía mucho (en el rango 1 a 125), pero la primera casi nada en términos relativos (rango 1 a 5). Esto lleva a que la varianza asociada a la primera coordenada, o eje x, es 2078, mientras que la varianza asociada a la segunda coordenada es 2 (elementos diagonales de la matriz de covarianza). Por lo tanto, una línea que explique bien la varianza de la primera coordenada (es decir, una componente principal alineada con el eje x) explicará una porción muy significativa de la varianza total.

Parte 3:

La norma Frobenius del residuo es igual a la raíz cuadrada de la suma de los valores propios no considerados, sobre la suma de todos los valores propios:

$$\|R\|_F = \sqrt{\frac{\lambda_3}{\lambda_1 + \lambda_2 + \lambda_3}} = \sqrt{\frac{0}{2080}} = 0$$

En este caso, el resultado es exactamente cero por que los datos, a pesar de ser tridimensionales, son linealmente dependientes y están -exactamente- en un plano (el plano z=1). Por tanto, alcanza con dos componentes principales (un subespacio de dimensión 2 = un plano) para describir la totalidad de los datos, sin errores.